

CS3014 Estructuras de Datos Avanzadas

Laboratorio - Semana 10 - Sesión 2

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

29 de mayo de 2026

¿Qué aprenderemos hoy?



¿Qué aprenderemos hoy?



► Strings

¿Qué aprenderemos hoy?



- ▶ Strings
- ▶ Suffix array

Strings

Notación



Notación básica de strings

Notación



Notación básica de strings

Una cadena es una secuencia de símbolos llamados *caracteres*, los cuales pertenecen a un conjunto llamado **alfabeto** Σ .

Notación



Notación básica de strings

Una cadena es una secuencia de símbolos llamados *caracteres*, los cuales pertenecen a un conjunto llamado **alfabeto** Σ .

El i -ésimo carácter de una cadena S se denota por $S[i]$ y la concatenación de los caracteres de las posiciones $x \in [i, j]$ se denotar por $S[i, j]$.

Notación



Notación básica de strings

Una cadena es una secuencia de símbolos llamados *caracteres*, los cuales pertenecen a un conjunto llamado **alfabeto** Σ .

El i -ésimo carácter de una cadena S se denota por $S[i]$ y la concatenación de los caracteres de las posiciones $x \in [i, j]$ se denotar por $S[i, j]$.

Convenciones

Notación



Notación básica de strings

Una cadena es una secuencia de símbolos llamados *caracteres*, los cuales pertenecen a un conjunto llamado **alfabeto** Σ .

El i -ésimo carácter de una cadena S se denota por $S[i]$ y la concatenación de los caracteres de las posiciones $x \in [i, j]$ se denotar por $S[i, j]$.

Convenciones

- Se considera comparación **lexicográfica** entre cadena.

Notación



Notación básica de strings

Una cadena es una secuencia de símbolos llamados *caracteres*, los cuales pertenecen a un conjunto llamado **alfabeto** Σ .

El i -ésimo carácter de una cadena S se denota por $S[i]$ y la concatenación de los caracteres de las posiciones $x \in [i, j]$ se denotar por $S[i, j]$.

Convenciones

- ▶ Se considera comparación **lexicográfica** entre cadena.
- ▶ Las cadenas deben tener como último carácter a un sentinela $\$ \in \Sigma$.
Dependiendo del caso, se podría asumir también que $\$ < c, \forall c \in \Sigma$.

Suffix array

Suffix array



Definición

Suffix array



Definición

El suffix array de una cadena S con longitud n es una permutación A de sus posiciones de manera que si i va antes que j en A , entonces $S[i, n - 1] < S[j, n - 1]$.

Suffix array



Definición

El suffix array de una cadena S con longitud n es una permutación A de sus posiciones de manera que si i va antes que j en A , entonces $S[i, n - 1] < S[j, n - 1]$.

Denotaremos por $\text{Suf}(S, i)$ al sufijo de S que comienza en la posición i , es decir, $S[i, n - 1]$.

Setup básico

Suffix array



Definición

El suffix array de una cadena S con longitud n es una permutación A de sus posiciones de manera que si i va antes que j en A , entonces $S[i, n - 1] < S[j, n - 1]$.

Denotaremos por $\text{Suf}(S, i)$ al sufijo de S que comienza en la posición i , es decir, $S[i, n - 1]$.

Setup básico

En primer lugar, para cualquier cadena S que tengamos como argumento, vamos a considerar que esta tiene concatenado un caracter sentinela, el cual es lexicográficamente menor que todos los caracteres del alfabeto.

Suffix array



Definición

El suffix array de una cadena S con longitud n es una permutación A de sus posiciones de manera que si i va antes que j en A , entonces $S[i, n - 1] < S[j, n - 1]$.

Denotaremos por $\text{Suf}(S, i)$ al sufijo de S que comienza en la posición i , es decir, $S[i, n - 1]$.

Setup básico

En primer lugar, para cualquier cadena S que tengamos como argumento, vamos a considerar que esta tiene concatenado un caracter sentinela, el cual es lexicográficamente menor que todos los caracteres del alfabeto. Esto nos ayuda evitando la cadena vacía.

Construcción del suffix array



La siguiente tabla muestra algunos algoritmos de construcción de suffix array (SACA) importantes históricamente.

Construcción del suffix array



La siguiente tabla muestra algunos algoritmos de construcción de suffix array (SACA) importantes históricamente.

Autor(es)	Algoritmo	Complejidad	Idea
Manber & Myers	Sin nombre	$O(n \log n)$	Doubling
Itoh & Tanaka	Sin nombre	Superlineal	Ordenamiento inducido
Ko & Aluru	SAKA	$O(n)$	Ordenamiento inducido
Karkkainen et. al.	DC3	$O(n)$	Ordenamiento inducido
Nong et. al.	SAIS	$O(n)$	Ordenamiento inducido

Cuadro: SACA más conocidos

DC3 (Karkkainen et. al.)



Idea

DC3 (Karkkainen et. al.)



Idea

Particionamos las posiciones según su residuo respecto a 3.

DC3 (Karkkainen et. al.)



Idea

Particionamos las posiciones según su residuo respecto a 3.
Denotamos los tres conjuntos

$$\mathcal{S}_0 = \{0 \leq i \leq n-1 : i = 3k, \text{ para algún } k \in \mathbb{Z}\}$$

$$\mathcal{S}_1 = \{0 \leq i \leq n-1 : i = 3k+1, \text{ para algún } k \in \mathbb{Z}\}$$

$$\mathcal{S}_2 = \{0 \leq i \leq n-1 : i = 3k+2, \text{ para algún } k \in \mathbb{Z}\}$$

DC3 (Karkkainen et. al.)



Idea

Particionamos las posiciones según su residuo respecto a 3.
Denotamos los tres conjuntos

$$\mathcal{S}_0 = \{0 \leq i \leq n-1 : i = 3k, \text{ para algún } k \in \mathbb{Z}\}$$

$$\mathcal{S}_1 = \{0 \leq i \leq n-1 : i = 3k+1, \text{ para algún } k \in \mathbb{Z}\}$$

$$\mathcal{S}_2 = \{0 \leq i \leq n-1 : i = 3k+2, \text{ para algún } k \in \mathbb{Z}\}$$

Teorema (Karkkainen et. al.)

DC3 (Karkkainen et. al.)



Idea

Particionamos las posiciones según su residuo respecto a 3.
Denotamos los tres conjuntos

$$\mathcal{S}_0 = \{0 \leq i \leq n-1 : i = 3k, \text{ para algún } k \in \mathbb{Z}\}$$

$$\mathcal{S}_1 = \{0 \leq i \leq n-1 : i = 3k+1, \text{ para algún } k \in \mathbb{Z}\}$$

$$\mathcal{S}_2 = \{0 \leq i \leq n-1 : i = 3k+2, \text{ para algún } k \in \mathbb{Z}\}$$

Teorema (Karkkainen et. al.)

Si se sabe el orden de los sufijos en los elementos de $\mathcal{S}_1$ y $\mathcal{S}_2$, entonces se puede deducir el orden de todos los sufijos de S .

Algoritmo



Partición

Algoritmo



Partición

Para S_1 y S_2 , vamos a tomar los bloques de 3 elementos en adelante desde cada posición. Ya que podrían faltar elementos, se hace padding con sentinelas \$.

Algoritmo



Partición

Para $\mathcal{S}_1$ y $\mathcal{S}_2$, vamos a tomar los bloques de 3 elementos en adelante desde cada posición. Ya que podrían faltar elementos, se hace padding con sentinelas \$.

Por ejemplo, para “banana\$”, se tendrían los siguientes bloques:

$$\mathcal{S}_1 \rightarrow (ana)(na\$)$$

$$\mathcal{S}_2 \rightarrow (nan)(a\$\$)$$

Algoritmo



Partición

Para $\mathcal{S}_1$ y $\mathcal{S}_2$, vamos a tomar los bloques de 3 elementos en adelante desde cada posición. Ya que podrían faltar elementos, se hace padding con sentinelas \$.

Por ejemplo, para “banana\$”, se tendrían los siguientes bloques:

$$\mathcal{S}_1 \rightarrow (ana)(na\$)$$

$$\mathcal{S}_2 \rightarrow (nan)(a\$\$)$$

Notemos que es posible mapear cada uno de estos bloques a su *rank* entre todos los bloques en $O(n)$ usando radix sort.

Algoritmo



¿Y ahora qué hacemos?

Algoritmo



¿Y ahora qué hacemos?

Podemos reemplazar cada bloque por su respectivo rank y se terminan creando dos cadenas nuevas.

Algoritmo



¿Y ahora qué hacemos?

Podemos reemplazar cada bloque por su respectivo rank y se terminan creando dos cadenas nuevas.

$$S_1 \rightarrow R_1 = 12$$

$$S_2 \rightarrow R_2 = 30$$

Algoritmo



¿Y ahora qué hacemos?

Podemos reemplazar cada bloque por su respectivo rank y se terminan creando dos cadenas nuevas.

$$S_1 \rightarrow R_1 = 12$$

$$S_2 \rightarrow R_2 = 30$$

Si concatenamos estas dos cadenas nuevas, tendremos una cadena $R = R_1 R_2$ de longitud $\approx \frac{2n}{3}$.

Algoritmo



¿Y ahora qué hacemos?

Podemos reemplazar cada bloque por su respectivo rank y se terminan creando dos cadenas nuevas.

$$S_1 \rightarrow R_1 = 12$$

$$S_2 \rightarrow R_2 = 30$$

Si concatenamos estas dos cadenas nuevas, tendremos una cadena $R = R_1 R_2$ de longitud $\approx \frac{2n}{3}$.

Aplicando recursión

Algoritmo



¿Y ahora qué hacemos?

Podemos reemplazar cada bloque por su respectivo rank y se terminan creando dos cadenas nuevas.

$$S_1 \rightarrow R_1 = 12$$

$$S_2 \rightarrow R_2 = 30$$

Si concatenamos estas dos cadenas nuevas, tendremos una cadena $R = R_1 R_2$ de longitud $\approx \frac{2n}{3}$.

Aplicando recursión

Podemos usar recursión para obtener el suffix array de la cadena R , de esta manera obtendremos el orden relativo entre cada par de sufijos de R_1 y R_2 .

Algoritmo



¿Y como ordeno todo?

Algoritmo



¿Y como ordeno todo?

Vamos a tener escenarios en los cuales todo se va a reducir a comparar una cantidad $O(1)$ de caracteres y una comparación de ranks de los sufijos de S_1 y S_2 .

Algoritmo



¿Y como ordeno todo?

Vamos a tener escenarios en los cuales todo se va a reducir a comparar una cantidad $O(1)$ de caracteres y una comparación de ranks de los sufijos de $\mathcal{S}_1$ y $\mathcal{S}_2$.

- ▶ $x \in \mathcal{S}_0$ vs $y \in \mathcal{S}_1$: Podemos tomar el primer caracter de cada sufijo, de manera que la comparación se reduce a comparar los pares $(S[i], \text{rank}(\text{Suf}(S, i + 1)))$ para $i = x, y$.
Notemos que $x + 1 \in \mathcal{S}_1$ y $y + 1 \in \mathcal{S}_2$, así que sus ranks están definidos.
- ▶ $x \in \mathcal{S}_0$ vs $y \in \mathcal{S}_2$: Podemos tomar el primer caracter de cada sufijo, de manera que la comparación se reduce a comparar las tuplas $(S[i], S[i + 1], \text{rank}(\text{Suf}(S, i + 2)))$ para $i = x, y$.
Notemos que $x + 2 \in \mathcal{S}_2$ y $y + 2 \in \mathcal{S}_1$, así que sus ranks están definidos.

Complejidad



Análisis

Complejidad



Análisis

Ya que es posible hacer la comparación de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma:

Complejidad



Análisis

Ya que es posible hacer la comparación de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma:

$$T(n) = T\left(\frac{2n}{3}\right) + O(n)$$

Complejidad



Análisis

Ya que es posible hacer la comparación de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma:

$$T(n) = T\left(\frac{2n}{3}\right) + O(n)$$

Es sencillo notar que $T(n) = O(n)$.

Complejidad



Análisis

Ya que es posible hacer la comparación de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma:

$$T(n) = T\left(\frac{2n}{3}\right) + O(n)$$

Es sencillo notar que $T(n) = O(n)$.

Generalización

Complejidad



Análisis

Ya que es posible hacer la comparación de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma:

$$T(n) = T\left(\frac{2n}{3}\right) + O(n)$$

Es sencillo notar que $T(n) = O(n)$.

Generalización

Se planteó en investigaciones posteriores la posibilidad de tomar módulo k en vez de módulo 3.

Complejidad



Análisis

Ya que es posible hacer la comparación de pares y tuplas en $O(n)$ usando radix sort, obtenemos una recurrencia de la siguiente forma:

$$T(n) = T\left(\frac{2n}{3}\right) + O(n)$$

Es sencillo notar que $T(n) = O(n)$.

Generalización

Se planteó en investigaciones posteriores la posibilidad de tomar módulo k en vez de módulo 3.

El módulo $k = 7$ es considerado uno de los mejores rendimientos.

SAIS (Nong et. al.)



Idea

SAIS (Nong et. al.)



Idea

Clasificar los sufijos de S en tipo S y L .

SAIS (Nong et. al.)



Idea

Clasificar los sufijos de S en tipo S y L .

Un sufijo $\text{Suf}(S, i)$ es de tipo

- ▶ S , si $\text{Suf}(S, i) < \text{Suf}(S, i + 1)$.
- ▶ L , si $\text{Suf}(S, i) > \text{Suf}(S, i + 1)$.

SAIS (Nong et. al.)



Idea

Clasificar los sufijos de S en tipo S y L .

Un sufijo $\text{Suf}(S, i)$ es de tipo

- ▶ S , si $\text{Suf}(S, i) < \text{Suf}(S, i + 1)$.
- ▶ L , si $\text{Suf}(S, i) > \text{Suf}(S, i + 1)$.

Un sufijo $\text{Suf}(S, i)$ es de tipo LMS si el sufijo $\text{Suf}(S, i - 1)$ es de tipo L y el sufijo $\text{Suf}(S, i)$ es de tipo S .

SAIS (Nong et. al.)



Idea

Clasificar los sufijos de S en tipo S y L .

Un sufijo $\text{Suf}(S, i)$ es de tipo

- ▶ S , si $\text{Suf}(S, i) < \text{Suf}(S, i + 1)$.
- ▶ L , si $\text{Suf}(S, i) > \text{Suf}(S, i + 1)$.

Un sufijo $\text{Suf}(S, i)$ es de tipo LMS si el sufijo $\text{Suf}(S, i - 1)$ es de tipo L y el sufijo $\text{Suf}(S, i)$ es de tipo S .

Notemos que hay a lo mucho $\frac{n-1}{2}$ sufijos de tipo LMS.

Teorema (Nong et. al.)

SAIS (Nong et. al.)



Idea

Clasificar los sufijos de S en tipo S y L .

Un sufijo $\text{Suf}(S, i)$ es de tipo

- ▶ S , si $\text{Suf}(S, i) < \text{Suf}(S, i + 1)$.
- ▶ L , si $\text{Suf}(S, i) > \text{Suf}(S, i + 1)$.

Un sufijo $\text{Suf}(S, i)$ es de tipo LMS si el sufijo $\text{Suf}(S, i - 1)$ es de tipo L y el sufijo $\text{Suf}(S, i)$ es de tipo S .

Notemos que hay a lo mucho $\frac{n-1}{2}$ sufijos de tipo LMS.

Teorema (Nong et. al.)

Si se sabe el orden relativo de los sufijos de tipo LMS, entonces se puede inducir el orden de todos los sufijos de S .

Algoritmo



Partición

Algoritmo



Partición

Se realiza la partición de S tomando en cuenta las posiciones de los sufijos LMS y se ordenan según $(\text{caracter}, \text{tipo})$ para calcular el rank. Es posible realizar esto en $O(n)$.

Algoritmo



Partición

Se realiza la partición de S tomando en cuenta las posiciones de los sufijos LMS y se ordenan según $(\text{caracter}, \text{tipo})$ para calcular el rank. Es posible realizar esto en $O(n)$.

Reducción

Algoritmo



Partición

Se realiza la partición de S tomando en cuenta las posiciones de los sufijos LMS y se ordenan según $(\text{caracter}, \text{tipo})$ para calcular el rank. Es posible realizar esto en $O(n)$.

Reducción

Se obtiene una cadena reducida R en base a la partición por LMS, se calcula el suffix array de R y se puede inducir el orden de todos los sufijos de S .

Algoritmo



Partición

Se realiza la partición de S tomando en cuenta las posiciones de los sufijos LMS y se ordenan según $(\text{caracter}, \text{tipo})$ para calcular el rank. Es posible realizar esto en $O(n)$.

Reducción

Se obtiene una cadena reducida R en base a la partición por LMS, se calcula el suffix array de R y se puede inducir el orden de todos los sufijos de S .

Complejidad

Algoritmo



Partición

Se realiza la partición de S tomando en cuenta las posiciones de los sufijos LMS y se ordenan según $(\text{caracter}, \text{tipo})$ para calcular el rank. Es posible realizar esto en $O(n)$.

Reducción

Se obtiene una cadena reducida R en base a la partición por LMS, se calcula el suffix array de R y se puede inducir el orden de todos los sufijos de S .

Complejidad

Ya que $|R| \leq \frac{n}{2}$, se tiene que

$$T(n) = T\left(\frac{n}{2}\right) + O(n) \rightarrow T(n) = O(n)$$

Calculando el LCP



LCP array

Calculando el LCP



LCP array

Es un arreglo en el cual se almacenan los Longest Common Prefix (LCP) entre cada par de posiciones adyacentes del suffix array

Calculando el LCP



LCP array

Es un arreglo en el cual se almacenan los Longest Common Prefix (LCP) entre cada par de posiciones adyacentes del suffix array

Ventajas

Calculando el LCP



LCP array

Es un arreglo en el cual se almacenan los Longest Common Prefix (LCP) entre cada par de posiciones adyacentes del suffix array

Ventajas

- ▶ Ya que a está ordenado, nos permite calcular el LCP entre cualquier par de posiciones en un tiempo prudente si nos apoyamos en alguna estructura de datos.

Calculando el LCP



LCP array

Es un arreglo en el cual se almacenan los Longest Common Prefix (LCP) entre cada par de posiciones adyacentes del suffix array

Ventajas

- ▶ Ya que a está ordenado, nos permite calcular el LCP entre cualquier par de posiciones en un tiempo prudente si nos apoyamos en alguna estructura de datos.
- ▶ Ya que a está ordenado, nos permite buscar las ocurrencias de cualquier cadena t en $O(|t| \log n)$.

Calculando el LCP



¿Y cómo se calcula?

Calculando el LCP



¿Y cómo se calcula?

Una implementación ingenua podría tomar hasta $O(n^2)$ de complejidad, así que usaremos el algoritmo de Kasai para calcularlo en un mejor tiempo.

Calculando el LCP



¿Y cómo se calcula?

Una implementación ingenua podría tomar hasta $O(n^2)$ de complejidad, así que usaremos el algoritmo de Kasai para calcularlo en un mejor tiempo.

Algoritmo de Kasai

Calculando el LCP



¿Y cómo se calcula?

Una implementación ingenua podría tomar hasta $O(n^2)$ de complejidad, así que usaremos el algoritmo de Kasai para calcularlo en un mejor tiempo.

Algoritmo de Kasai

Su principal observación es que si tenemos dos sufijos que son adyacentes en a , sean x y y sus posiciones iniciales, con LCP igual a k , entonces $x + 1$ y $y + 1$ tienen LCP de al menos $k - 1$.

Calculando el LCP



¿Y cómo se calcula?

Una implementación ingenua podría tomar hasta $O(n^2)$ de complejidad, así que usaremos el algoritmo de Kasai para calcularlo en un mejor tiempo.

Algoritmo de Kasai

Su principal observación es que si tenemos dos sufijos que son adyacentes en a , sean x y y sus posiciones iniciales, con LCP igual a k , entonces $x + 1$ y $y + 1$ tienen LCP de al menos $k - 1$.

Para recorrer de esa manera los x , se itera de sufijo de mayor longitud a sufijo de menor longitud.

Calculando el LCP



Pero podrían no ser adyacentes

Calculando el LCP



Pero podrían no ser adyacentes

Eso es cierto, pero si su LCP es al menos $k - 1$, eso implica que todas las posiciones entre ellos también tienen LCP de al menos $k - 1$ con cualquiera de los dos.

Calculando el LCP



Pero podrían no ser adyacentes

Eso es cierto, pero si su LCP es al menos $k - 1$, eso implica que todas las posiciones entre ellos también tienen LCP de al menos $k - 1$ con cualquiera de los dos.

Complejidad

Calculando el LCP



Pero podrían no ser adyacentes

Eso es cierto, pero si su LCP es al menos $k - 1$, eso implica que todas las posiciones entre ellos también tienen LCP de al menos $k - 1$ con cualquiera de los dos.

Complejidad

Si k se reduce en 1 cada vez que comparamos un par de posiciones de a , este es reducido a lo mucho $n - 1$ veces (ya que la última posición no tiene un sucesor). Además, k debe permanecer en el rango $[0, n]$, así que si usamos una iteración ingenua para cada x y y , tendremos una complejidad de $O(n)$.

Calculando el LCP



Suffix	Index	LCP
\$	6	–
A\$	5	0
ANA\$	3	1
ANANA\$	1	3
BANANA\$	0	0
NA\$	4	0
NANA\$	2	2

Propiedades



RMQ

Propiedades



RMQ

Se puede calcular el LCP entre un par de sufijos de S mediante una consulta de RMQ sobre el LCP array.

Propiedades



RMQ

Se puede calcular el LCP entre un par de sufijos de S mediante una consulta de RMQ sobre el LCP array.

(Spoiler) El suffix tree es equivalente al auxiliary/tree de todos los sufijos y el LCA representa al LCP entre cada par de hojas.

Propiedades



RMQ

Se puede calcular el LCP entre un par de sufijos de S mediante una consulta de RMQ sobre el LCP array.

(Spoiler) El suffix tree es equivalente al auxiliary/tree de todos los sufijos y el LCA representa al LCP entre cada par de hojas.

Conclusiones

Propiedades



RMQ

Se puede calcular el LCP entre un par de sufijos de S mediante una consulta de RMQ sobre el LCP array.

(Spoiler) El suffix tree es equivalente al auxiliary/tree de todos los sufijos y el LCA representa al LCP entre cada par de hojas.

Conclusiones

Los suffix array son estructuras poderosas que permiten obtener casi tanta información como otras estructuras (suffix tree, suffix automaton) pero normalmente usa menos memoria, lo cual es una ventaja.

Resumen de la sesión



Resumen de la sesión



- ▶ Aprendimos suffix array

Resumen de la sesión



- ▶ Aprendimos suffix array
- ▶ Aprendimos el algoritmo DC3 y SAIS

Resumen de la sesión



- ▶ Aprendimos suffix array
- ▶ Aprendimos el algoritmo DC3 y SAIS
- ▶ Aprendimos LCP array

Gracias